HelixCode — Open Issues Tracker

> Per Constitution §11.4.15 (Item-status tracking) + §11.4.16 (Item-type tracking) + §11.4.19 (Fixed-document column-alignment) + CONST-057 (Type-aware closure vocabulary) + CONST-058 (Reopened-source attribution).
>
> **Authoritative resumption ledger**: `docs/CONTINUATION.md` (CONST-044). This file complements it with item-level granularity for currently-open work.
>
> **Status vocabulary** (closed set): `Queued` | `In progress` | `Ready for testing` | `In testing` | `Reopened` | `Fixed/Implemented/Completed (→ Fixed.md)`
>
> **Type vocabulary** (closed set): `Bug` | `Feature` | `Task`

Prefix convention

Round 189 (2026-05-19) introduced per-project / per-submodule ID prefixes replacing the legacy `ISSUE-NNN` flat namespace. New items MUST use the prefix matching their scope; cross-cutting items affecting two or more submodules (or any meta-repo concern) live under `HXC`. Numeric portion is per-prefix (each prefix starts at `001`). Legacy `ISSUE-NNN` IDs renamed forward-only per CONST-043; historical close-out narratives in `docs/CONTINUATION.md` preserve original IDs verbatim, and `docs/Fixed.md` annotates each closure with `(ex-ISSUE-NNN)` for git-history traceability.

Prefix	Scope	Source
HXC	HelixCode root project (project-wide, multi-submodule,	this repo

Prefix	Scope	Source
	governance, infrastructure)	
HXA	HelixAgent submodule	`submodules/helix_agent` (when present) / `helix_agent/` tree
HXM	HelixMemory submodule	`submodules/helix_memory`
HXL	HelixLLM submodule	`submodules/helix_llm`
HXQ	HelixQA submodule	`submodules/helix_qa` (`helix_qa/`)
HXS	HelixSpecifier submodule	`submodules/helix_specifier`
HXO	HelixOrchestrator (= LLMOrchestrator) submodule	`submodules/llm_orchestrator`
HXV	HelixVerifier (= LLMsVerifier) submodule	`submodules/llms_verifier`
HXD	HelixDocProcessor (= DocProcessor) submodule	`submodules/doc_processor`
HXI	HelixI18n (when added)	tba
PLN	Planning submodule (vasic-digital)	`submodules/planning`
VEN	VisionEngine submodule (HelixDevelopment)	`submodules/vision_engine`
SLF	SelfImprove submodule (vasic-digital)	`submodules/self_improve`
STO	Storage submodule (vasic-digital)	`submodules/storage`
OPS	LLMOps submodule (vasic-digital)	`submodules/llm_ops`

Prefix	Scope	Source
VDB	VectorDB submodule (vasic-digital)	`submodules/vector_db`
OBS	Observability submodule (vasic-digital)	`submodules/observability`
MCP	MCP_Module submodule (vasic-digital)	`submodules/mcp_module`
MSG	Messaging submodule (vasic-digital)	`submodules/messaging`
MDW	Middleware submodule (vasic-digital)	`submodules/middleware`
PLG	Plugins submodule (vasic-digital)	`submodules/plugins`
STR	Streaming submodule (vasic-digital)	`submodules/streaming`
WAT	Watcher submodule (vasic-digital)	`submodules/watcher`
CNV	conversation submodule (vasic-digital)	`submodules/conversation`
AUT	Auth submodule (vasic-digital)	`submodules/auth`
LZY	Lazy submodule (vasic-digital)	`submodules/lazy`
ATP	AutoTemp submodule (vasic-digital)	`submodules/auto_temp`
PLI	PliniusCommon submodule (vasic-digital)	`submodules/plinius_common`
CHL	challenges submodule (vasic-digital)	`challenges/`
CNT		`containers/`

Prefix	Scope	Source
	containers submodule	
SEC	security submodule	`security/`
PAN	panoptic submodule	`panoptic/`

For submodules not listed above, default to the first 3 letters of the submodule name, uppercase (e.g. `Watcher` → `WAT`). Document the new prefix in this table on first use.

Legacy → new mapping (round 189)

Old ID	New ID	Scope rationale
ISSUE-001	VEN-001	VisionEngine `helix-gitlab` URL
ISSUE-002	VEN-002	VisionEngine `vasic-digital-github` fork divergent
ISSUE-003	HXL-001	HelixLLM `analysis_test.go` hardcoded path
ISSUE-004	HXL-002	HelixLLM TOON `WriteTOON` 500
ISSUE-005	HXC-001	CONST-052 rename programme (project-wide)
ISSUE-006	HXC-002	Round-74 residual LOGIC FAILs (multi-submodule cross-cutting)
ISSUE-007	HXC-003	CONST-046 migration backlog (project-wide)
ISSUE-008	HXQ-001	helix_qa `TestPerformance` flake
ISSUE-009	HXA-001	helix_agent handler tests (4)
ISSUE-010	HXA-002	helix_agent debate API drift
ISSUE-011	HXA-003	venice CONST-037 (in helix_agent)

HXC-159 — Fully incorporate the HelixSkills System into HelixCode + HelixAgent (all power-features, SpecKit-driven)

Status: Queued **Type:** Task **Severity:** High **Created-By:** Operator

Reported-Via: §11.4.202 reporting directive `task` on 2026-07-28T19:26:22Z **Reported-By:** Operator

What (the report, verbatim): Feature description (verbatim): Fully incorporate the HelixSkills System (git@github.com:HelixDevelopment/skills.git) into BOTH HelixCode and HelixAgent, covering ALL of its power-features with nothing left out.

VERBATIM OPERATOR REQUIREMENTS (2026-07-29): “Fully incorporate into HelixCode and HelixAgent the HelixSkills System for all work with Skills: git@github.com:HelixDevelopment/skills.git , we MUST fully incorporate all HelixSkills power-features fully without nothing leftout! Full exhaustive implementation plan divided into phases, tasks and subtasks with exhaustive amount of data with all details MUST BE prepared! Full coverage with ll supported test types, challenges and HelixQA test banks (suites) is mandatory! Extend and update all existing documentation, user manuals, guides, FAQs, create stunning illustrations, graphs and diagrams and incorporate them in all materials! Any gaps, shortcomings, weak spots, danger zones, issues, or any inconsistencies MUST BE detected in advance during the planning and implementation process and properly tackled with comprehensive rock-solid solutions implementations, risk-free and safe! Track all progress through the constinuation docs, memory, knowledge and remebering mechanisms we have! Use GitHub SpecKit for all phases of incorporating with bridge to Superpowers used for the implementation and subagents driven development mechanism!”

PRE-VERIFIED ENVIRONMENT FACTS (established 2026-07-29, not assumed — §11.4.6): - GitHub SpecKit IS installed at /home/milos/.local/bin/specify. It is NOT yet initialized in this repo (no .specify/ and no specs/ directory present). Initialization is therefore an explicit early task, not an assumption. - git@github.com:HelixDevelopment/skills.git IS reachable via SSH. `git ls-remote --heads` returns branches including feature/catalog-docs, feature/deep-research, feature/testing-infra. The default branch and full branch/tag inventory MUST be re-derived at research time rather than inferred from this partial listing. - The repository is NOT currently a submodule of this project. `.gitmodules` contains no HelixDevelopment/skills entry; the only ‘skills’-matching entry is cli_agents/codex-skills -> vasic-digital/caf-codex-skills.git, which is a DIFFERENT repository and must not be confused with it. - Both consumers exist in-tree: the inner Go module helix_code/ (module dev.helix.code) and the submodule submodules/helix_agent (module dev.helix.agent).

MANDATORY DELIVERY CONSTRAINTS: - SpecKit-first: use GitHub SpecKit for ALL phases of the incorporation, bridged to Superpowers for implementation, executed subagent-driven (§11.4.70 / §11.4.20). - Exhaustive plan structured as phases -> tasks -> subtasks, with an exhaustive level of detail (down to lines-of-code and micro-POCs where the design is load-bearing). - Complete test coverage per §11.4.169: unit, integration, e2e, full-automation, Challenges (vasic-digital/challenges), HelixQA banks (HelixDevelopment/helix_qa) with autonomous sessions, DDoS, security, stress + chaos, concurrency/atomicity, race/deadlock, memory, benchmarking. Every PASS carries captured evidence (§11.4.5 / §11.4.69); every guard ships a paired §1.1 mutation. - Documentation: extend AND update every existing doc, user manual, guide and FAQ — not only new ones. Produce illustrations, graphs and diagrams and incorporate them into all materials. Four-format export discipline per §11.4.65 / §11.4.153. Every doc reachable from the main README per §11.4.212. - Risk work is a PLANNING deliverable, not a post-hoc note: every gap, shortcoming, weak spot, danger zone, issue and inconsistency MUST be detected DURING planning and each one paired with a comprehensive, rock-solid, risk-free and safe solution (§11.4.102 systematic-debugging over all gathered data). - Reuse before rewrite (§11.4.74 / §11.4.28): survey the vasic-digital and HelixDevelopment catalogues on GitHub AND GitLab first; extend owned submodules in place rather than duplicating, keeping them project-agnostic and decoupled. HelixSkills itself must be incorporated as a properly-laid-out dependency per §11.4.28(C) / CONST-051(C) — root-level or submodules/, never a nested own-org chain. - Cross-consumer parity: HelixCode and HelixAgent must BOTH receive the full feature set; a capability landing in one and not the other is an incompleteness to be tracked, not silently accepted. - Progress tracked through the continuation docs, the memory/knowledge/remembering mechanisms, and the workable-items SSoT (§12.10 / §11.4.131 / §11.4.93 / §11.4.95).

ACCEPTANCE CRITERIA: 1. A SpecKit specification exists and is initialized in-repo, covering every HelixSkills power-feature, with an explicit feature inventory proven complete against the upstream repository (an enumerated list, not a sample — §11.4.118). 2. A phase/task/subtask implementation plan exists at the stated depth, with each risk item paired to a concrete mitigation. 3. Every planned capability is wired and end-user-reachable in BOTH consumers, proven by §11.4.108 runtime signatures on a clean target — not merely compiled. 4. The full §11.4.169 test-type matrix is green with captured evidence, Challenges and HelixQA banks included. 5. All documentation is updated and exported, with the produced diagrams/illustrations

incorporated, and reachable from README. 6. Independent code review reaches a zero-finding GO per §11.4.125 / §11.4.134 / §11.4.142. 7. No capability is claimed without runtime evidence — the anti-bluff covenant governs every closure (§11.4 / §11.4.1 / §11.4.123).

§11.4.213 FEATURE research-and-planning WORK PROGRAM (scheduled, not yet executed):

1. Deep web-research series (articles / guides / scientific papers / open-source projects+codebases / real-world examples) on the best way to incorporate this feature into the project (§11.4.8 / §11.4.99 / §11.4.150).
2. Systematic-debug (§11.4.102) of ALL data obtained to enumerate EVERY weak spot / gap / danger-zone / inconsistency / imperfection, and design a risk-free, rock-solid solution for EACH one found.
3. The design MUST ALWAYS be enterprise-grade, bleeding-edge, and innovative – never less.
4. MANDATORY exhaustive documentation: many technical documents, an implementation plan down to lines-of-code + micro-proof-of-concepts, diagrams / schemes / graphs, illustrations, SQL definitions, templates, and every other relevant material (§11.4.65 / §11.4.73).
5. Investigate + plan REUSE of existing vasic-digital + HelixDevelopment submodules/components BEFORE proposing a rewrite; extend them freely where genuinely needed while keeping them fully decoupled / project-not-aware / reusable (§11.4.28 / §11.4.74 / §11.4.177).
6. Plan from the TESTING point of view from day one: every supported test type, the Challenges submodule, and full HelixQA bank coverage (§11.4.27 / §11.4.169).
7. Divide the work into phases / tasks / subtasks, fine-grained and nano-detailed enough to drive integration / implementation / wiring / testing / scaling directly.
8. Plan explicitly for enterprise scalability and maximal performance.
9. When the feature has a UI/UX surface, produce full wireframes / diagrams / design files (Figma / PSD / PDF, or whatever the project mandates) authored via OpenDesign (§11.4.162 / §11.4.190).
10. Plan full CodeGraph integration with regular / real-time index synchronisation (§11.4.78 / §11.4.79 / §11.4.80).
11. Create fully-detailed follow-on workable items (every detail + reference + attachment) synced in REAL TIME to the SQLite single-source-of-truth (§11.4.93 / §11.4.95) + every derived workable-items document + every related project doc/component + every connected external work-tracking system (§11.4.148 D5) – honestly SKIPPING any absent tracker with a machine-readable reason

(credentials_absent / tracker_client_absent, §11.4.10) rather than EVER faking a push.

Research-doc destination (to be populated when this item is worked): docs/research/fully_incorporate_the_helixskills_system_into_helixcode_heli_20260728T192622Z_2557683/

Execution model (§11.4.213 clauses 1-2): this item is SCHEDULED, not yet executed. The multi-day research/planning/documentation work above is performed LATER by the project's standing autonomous loop (§11.4.87 / §11.4.94 / §11.4.97 / §11.4.103 / §11.4.126) when it claims this item, and MUST be driven to a genuinely COMPLETED-and-wired or explicitly evidence-backed CLOSED terminal state under §11.4.197 – this item MUST NOT be left sitting un-wired in the backlog.

Affected scope / file-scope manifest: Feature research/planning – destination: docs/research/fully_incorporate_the_helixskills_system_into_helixcode_heli_20260728T192622Z_2557683/

Reproduction / context: UNKNOWN: not stated in the report — a reproduction MUST be established before any fix (§11.4.102 / §11.4.146 / §11.4.199)

Acceptance criteria: The full research-doc tree exists under docs/research/fully_incorporate_the_helixskills_system_into_helixcode_heli_20260728T192622Z_2557683/, every enumerated weak-spot/gap/danger-zone carries a designed risk-free solution, an implementation plan down to lines-of-code exists, the planned follow-on workable items are created, and the whole effort is validated per §11.4.197 (never left un-wired).

=== ADDENDUM — BINDING OPERATOR REQUIREMENT (2026-07-29, added after scheduling) ===

VERBATIM: "Make sure that SpecKit and Superpowers are used with Bridge extension and with all extensons we have created derrived from constitution Submodule!"

This is not optional colour on the SpecKit mandate — it names specific existing assets that this work MUST compose with rather than duplicate (§11.4.74 extend-don't-reimplement).

THE BRIDGE EXTENSION (verified to exist, 2026-07-29): constitution/docs/research/extensions/speckit_superpowers/implementation/ — the "Helix Constitution-Powered SpecKit-Superpowers Bridge". Eight

documents, each in .md/.html/.pdf/.docx: README, IMPLEMENTATION_PLAN, NANO_TASK_ENGINE, EXTENSION_DEVELOPMENT, TDD_INTEGRATION, CONSTITUTION_INTEGRATION, SECURITY, APPENDIX.

It defines a SEVEN-LAYER architecture: 1 Developer Workstation 2 Spec-Kit Core (governance) 3 SuperSpec bridge (orchestration — github.com/WangX0111/superspec) 4 SuperB extension (discipline enforcement — speckit-community.github.io/extensions/superb) 5 SuperBridge MCP (execution) 6 Helix LLM (inference — github.com/HelixDevelopment/helix_llm) 7 Distributed host cluster (llama.cpp RPC)

CONSTITUTION-DERIVED EXTENSIONS THAT MUST BE IN SCOPE (inherited BY REFERENCE per §11.4.228 — never copied locally, a copy diverges silently): constitution/skills/ (7) action-prefix-system, media-validator, multitrack, reporting-workable-items, scheduled-work-queue, session-sync, workable-item-lifecycle constitution/mcp/ (2) media-validator-mcp.json, scheduled-work-mcp.json constitution/plugins/ (2) helix, scheduled-work constitution/actions/ registry.yaml (§11.4.140 action system), subagent_tiering.yaml constitution/scripts/hooks/ (7 live) action_prefix_expand.sh, credential_scan_lib.sh, guard-branch-consistency.sh, guard-forbidden-commands.sh, guard-track-branch-label.sh, guard-work-track-binding.sh, post-merge

ALREADY PRESENT LOCALLY — do not re-create: .claude/skills/ 10 registered speckit-* skills (analyze, checklist, clarify, constitution, converge, implement, plan, specify, tasks, taskstoissues) .specify/workflows/speckit/workflow.yml, integrations/{claude,speckit}.manifest.json, scripts/bash/, templates/, memory/

ADDED ACCEPTANCE CRITERIA: 8. Every constitution-derived extension above has an explicit disposition — reused, extended, or orthogonal-with-reason. Silence is not an answer (§11.4.118). 9. Each of the 7 Bridge layers is classified WIRED vs DESIGNED-ONLY with a captured probe. A design document existing is NOT evidence a layer is installed — that is precisely the §11.4.108 SOURCE-vs-RUNTIME gap, and reporting a documented layer as an available one would be a §11.4 bluff. 10. The skill-namespace collision risk is addressed with a designed mitigation: HelixSkills is itself a skills system, while §11.4.164's post_update_hook.sh already registers constitution skills into .claude/skills/. Two systems registering into one namespace needs defined precedence and clash handling, not discovery at runtime. 11. SuperSpec and SuperB are THIRD-PARTY, non-own-org dependencies

(§11.4.74 vendor path) — their health, maintenance status and supply-chain exposure must be assessed, not assumed from the design doc referencing them.

HXC-164 — Infrastructure startup script drives containers directly instead of through the shared containers component

Status: Queued **Type:** Task **Created-By:** Claude **Severity:** Medium (violates §11.4.76 — bypasses the shared containers component and has already caused one real outage per the ticket's own account; remaining error-hiding on optional service startups makes a future failed start indistinguishable from success, a live anti-bluff gap — not High because no currently-shipped user path is broken today)

The script that brings up the supporting services for testing talks to the container tooling directly, rather than going through the shared containers component the organisation maintains for exactly this purpose. That component exists so every project starts services the same way, gets the same health checking, and benefits from fixes once rather than repeatedly. Bypassing it means this project carries its own copy of that logic, which drifts and has already caused one outage of its own. A related leftover is that several optional service startups still hide their errors, so a failure to start looks identical to a success. This was flagged honestly in a commit message, but a note in a commit message is not an obligation anyone will act on, which is why it is being tracked here. Closing this means routing the startup through the shared component and removing the remaining error-hiding so a failed start is visible.

HXC-166 — The agent component has 204 known security advisories against its dependencies, including 5 critical

Status: Queued **Type:** Bug **Created-By:** Claude **Severity:** Medium (triaged 2026-08-06: 0 of 210 live advisories are reachable-and-unmitigated in any shipped helix_agent artifact. Go: govulncheck from the 8 shipped cmd/ entrypoints reports "affected by 0 vulnerabilities"; the 3 symbol-reachable docker findings reach only internal/clis/openhands, which has 0 importers and 0 presence in the shipped

dependency closure. npm/pip: 148 of 210 (incl. 3 of 4 distinct criticals) are in mcp-servers/GitMCP, whose compose service names a Dockerfile that does not exist; 25 (incl. the handlebars critical) are in sdk/web, which no release mechanism builds. Exactly 1 alert (GHSA-w48q-cv73-mx4w) touches the genuinely shipped plugins/mcp-server, and it is already mitigated in code by HXC-172/d0a53e0b with paired tests. Medium not Low because mcp-gitmcp is a wired compose service one missing file away from building 3 criticals into a running container; not High because no reachable path to a shipped artifact exists today.)

When publishing the agent component, the hosting provider reported 204 outstanding security advisories affecting the libraries it depends on: 5 rated critical, 80 high, 100 moderate and 19 low. These are known, publicly documented weaknesses in third-party code the product ships and runs, so anyone who knows about them knows about them for our deployment too. Nothing here was introduced by recent work; the count has simply been accumulating and was surfaced by a routine publish. The risk is real but unquantified for us specifically, because a published advisory only matters if the affected code path is actually reachable in how we use the library. The right next step is to pull the full advisory list, separate the ones we genuinely reach from the ones we do not, and upgrade or replace the dependencies behind the critical and high findings first. Until that triage is done we cannot honestly claim the component is free of known vulnerabilities.

HXC-168 — A database password is written directly into published setup files and has never been changed

Status: Operator-blocked **Type:** Bug **Operator-Block-Details:** WHAT: Two DIFFERENT credentials are in play and the item’s original framing is now partly stale. (1) The literal HXC-168 was filed about (sha8=d297e142, 9 chars) was REMOVED from every tracked executable file by commit 11861996 (2026-07-31), which sourced it from a gitignored .env and added scripts/gates/ no_hardcoded_db_credential_gate.sh; that commit explicitly states it does NOT close HXC-168 because the value was never rotated. It survives only in docs/*.md|.html and one .txt. (2) The UNFIXED and more serious exposure is a SECOND literal (sha8=2087713a, 23 chars) at compose.helixcode-infra.yml:30 and again as an inline connection URL at :215 - tracked, and published on all four mirrors via commit f91d4477.*

Three independent reads confirm it is the LIVE database password: the tracked compose file, podman inspect of helixcode-infra-postgres .Config.Env, and the gitignored 0600 .env HELIX_DATABASE_PASSWORD - all sha8=2087713a. So the private-config mechanism exists and is wired, but the value inside it was never rotated away from the published one. Reachability is NOT localhost-only: ss -ltn shows LISTEN :5433 (wildcard bind, LAN-reachable on 10.6.100.0/24); the host has only RFC1918 addresses, so public-internet exposure is NOT established either way (no firewall tooling on PATH to verify - honest gap per 11.4.6). Mitigating: the database is helixcode_test, user helixcode. Two defects in the guard itself: no_hardcoded_db_credential_gate.sh currently exits 1 (CHECK 1 trips on the fix's own tracked evidence transcript, a .txt its prose exclusion misses), and compose.helixcode-infra.yml is NOT in its SCOPE_FILES - which is why it reports PASS on the file that boots the live platform. The gate is also not wired into any pre-build sweep, so nothing runs it automatically. WHY: Self-resolution exhausted for the ROTATION half only; the FILE half remains agent-safe and is enumerated as choice [E]. Rotation requires ALTER ROLE helixcode PASSWORD against the LIVE helixcode-infra-postgres (POSTGRES_PASSWORD applies only at volume init, so a compose edit alone changes nothing on an existing volume), rewriting .env, and recreating the postgres container plus every consumer - with three other agents live in this checkout and a green platform (gateway :8443, helixagent :7061, helixcode-server :8081, 12 containers, NRestarts=0). That is irreversible-in-effect, high-blast-radius, and its safe choice is not determinable from captured evidence, so 11.4.101 requires a BLOCK rather than an autonomous decision; 11.4.122 additionally forbids disturbing running components without operator confirmation. Because the value is already in git history on four mirrors, 11.4.113 forbids rewriting it away - rotation is the ONLY remedy, never a history rewrite. UNBLOCK: [A] ROTATE NOW - ALTER ROLE + regenerate .env + recreate helixcode-infra-postgres and dependent services; costs seconds-to-minutes of DB downtime and WILL disrupt the three live agents and the currently-green platform. [B] ROTATE AT THE NEXT MAINTENANCE WINDOW - no disturbance now; the published-credential exposure persists until then. [C] REDUCE EXPOSURE FIRST, ROTATE LATER - rebind 5433 (and autoboot 55432, docker/ 5432) to 127.0.0.1; container-restart only, no credential change; shrinks LAN reach to loopback immediately. [D] ACCEPT AND RECORD THE ACCEPTANCE (11.4.90 Obsolete with Obsolete-Details) on the grounds that the DB is helixcode_test on an RFC1918-only host - but this CANNOT be justified as 'localhost only': the bind is *:5433 and is LAN-reachable. [E] SPLIT THE ITEM - let an agent land the file half now (compose.helixcode-infra.yml:30,215; helix_code/

docker-compose.full-test.yml:24,473; helix_code/.env.full-test:15; helix_code/tests/e2e/.env.example:67; helix_code/test_programs/test_db_connection.go:15; the instruction-shaped doc copies) plus the two gate defects, with NO live impact, and keep a separate operator-owned rotation item - which is exactly how 25d41351 -> 11861996 already sequenced this work. WHO: Operator **Severity:** High **Created-By:** Claude

The password used to reach the project database is written in plain text inside several setup and container files, and those files are published on all four public code-hosting accounts. Anyone who reads the project can read the password. This is not new and was not introduced by recent work; the project's own earlier security review already recorded it as something that must be replaced, and that has still not happened. The practical risk depends on whether the same password is used anywhere reachable from outside, which has not been established either way and should not be assumed harmless. Because the value is already public, changing the files alone is not enough — the password itself has to be changed everywhere it is used, and the setup files must then read it from a private configuration source rather than containing it. Until both halves are done the project cannot honestly claim its credentials are protected.

HXC-171 — A copied-in third-party web app carries most of our reported security warnings but cannot even be built

Status: Queued **Type:** Bug **Severity:** Medium

A complete copy of somebody else's web application sits inside the agent's source tree, and it alone accounts for one hundred and thirty-nine of the two hundred and four reported dependency security warnings, including three of the five most severe. It cannot currently be built or run: the startup configuration points at a build recipe file that does not exist in that folder, and its supporting packages have never been downloaded here. This matters in two opposite ways at once. Because it is not built, those warnings do not describe anything we actually run today, which is why they rank low. But because a startup entry still references it, the setup is broken and misleading, and if anyone ever supplies the missing recipe file, all one hundred and thirty-nine warnings become live at once with no warning to whoever does it. Anyone reading our security reports benefits from removing this distortion, since it currently buries the small number of

warnings that genuinely matter. The expected outcome is an explicit decision, recorded in writing: either keep the copy and properly maintain and build it, or remove it and its broken startup entry, or replace it with a reference to the upstream hosted service. Because deleting shipped components requires approval, the decision must be put to the operator rather than taken unilaterally.

HXC-172 — Three security warnings in a small plugin service we wrote and actually deploy, with no reachability evidence either way

Status: Operator-blocked **Type:** Task **Severity:** High

A small companion service we wrote ourselves is packaged into a container and started as part of the standard deployment, and it carries three known security warnings in its supporting packages, two of them rated high. Unlike the great bulk of the reported warnings, these have no mitigating evidence in either direction: we cannot say the code is unshipped, because it demonstrably ships, and no analysis has been run to establish whether the affected functionality is ever exercised. This matters because it is the one place in the whole review where something we genuinely run carries unassessed risk, which makes it far more deserving of attention than its small count suggests. One of the three is not really an upgrade problem at all: it concerns a protective setting that is switched off unless explicitly enabled, so updating the package without also turning the protection on would leave the exposure in place while appearing to resolve it. Users of any deployment running this service benefit. The expected outcome is a reachability assessment of the three warnings, the two straightforward package updates applied, the protective setting explicitly verified as enabled in our configuration rather than assumed, and the service rebuilt and confirmed working.

HXC-175 — An unreachable safety fallback in the model cache could silently serve stale data forever if edited

Status: Queued **Type:** Bug **Severity:** Low **Created-By:** Claude

The cache that remembers model information has a small safety fallback for the case where its clock was never configured. That fallback can never actually run today, because the only way to build the cache always configures the clock — so it is untested and unexercised. On its own that is harmless. The reason it is worth recording is what happens if someone later edits it: a plausible-looking change to return an empty time instead of the current time would make every cached entry appear to have been fetched at a point so far in the past that the freshness check reads as a negative age, which never exceeds the expiry limit. Every entry would then be considered fresh forever, and the product would keep serving outdated model information indefinitely with no error and no failing test. The choice is to either add a small test that pins the fallback's behaviour, or remove the branch entirely and document that there is one supported way to construct the cache.

HXC-178 — The API key check compares secrets in a way that can leak them one character at a time

Status: Queued **Type:** Bug **Severity:** Low **Created-By:** Claude

When a caller presents an API key, the server compares it to the configured key using an ordinary text comparison. That kind of comparison stops as soon as it finds a character that does not match, so a key sharing the first few characters takes measurably longer to reject than one that differs immediately. An attacker who can measure those tiny differences across many attempts can recover the key one character at a time without ever guessing it outright. Network timing noise makes this difficult in practice, which is why it is low rather than urgent, but it is a genuine weakness and the remedy is standard and cheap: use the comparison function designed for secrets, which always takes the same amount of time regardless of where the difference falls. This is pre-existing and was not introduced by recent work.

HXC-179 — A browser-origin check safely allows requests with no origin, but only because a separate login check is wired in

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude

The check that decides which websites may open a live connection to our server deliberately allows requests that carry no origin information at all. That is correct behaviour, because only browsers send that information and non-browser clients legitimately omit it. It is safe today only because a separate login check runs first and turns away anyone without valid credentials. The two protections are therefore coupled, but nothing records or enforces that coupling: if someone later removed or reordered the login check while working on something unrelated, every client that omits origin information would become completely ungated, and no existing test would notice. The work here is to add a check that specifically confirms the login step is still wired ahead of the connection upgrade, so the dependency is protected rather than merely true by luck.

HXC-180 — Three historical commits cannot be built, and this must be documented because it can never be repaired

Status: Queued **Type:** Task **Severity:** Low **Created-By:** Claude

Three commits in the current release range do not build. All three share one cause: a test referred to a piece of data that was not added until a later commit, so anyone checking out those three points gets a build error. The tip of the branch is fine and the problem was corrected shortly afterwards. This cannot be repaired, because repairing it would mean rewriting published history, which the project forbids outright. The practical consequence is for anyone hunting a regression by stepping backwards through history: they will hit three points that fail to build for a reason unrelated to whatever they are hunting, and may wrongly conclude the fault lies there. The work here is simply to record the three affected points and their shared cause somewhere a person doing that search will find it, so the failure is recognised immediately as known and irrelevant rather than investigated from scratch.

HXC-181 — Amend the constitution canon's superseded summary-generator naming and re-cascade to the 129 owned-submodule copies

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude **Assigned-To:** Claude

HXC-160 corrected the 21 stale references to scripts/generate_issues_summary.sh and scripts/generate_fixed_summary.sh across the 12 parent-repo governance carriers, and added the CM-SUPERSEDED-GENERATOR-NAMING gate that guards them. Two reference sets were deliberately left untouched and are the subject of this item. First, constitution/Constitution.md still carries 14 references across several different anchors, including universal rules that projects other than HelixCode consume and historical migration-plan records such as the Phase 5 Go-binary-shim plan. Amending canonical constitutional text is a governance amendment that CONST-049 requires be committed and pushed to every configured upstream, then re-cascaded and pointer-bumped. Pushing was forbidden in the session that closed HXC-160, and a canon amended locally but never propagated is worse drift than the stale wording, so the canon was left alone. Second, 129 files across roughly 45 owned submodules carry cascaded restatements of the same two anchor sentences. The correction applied in the parent repo is deliberately scoped with the words in HelixCode, because the fix names a HelixCode-specific mechanism. Copying that HelixCode-scoped wording into decoupled, project-agnostic submodules would inject project context into them and violate CONST-051(B). The correct sequence is therefore to first agree project-neutral canonical wording that names the workable-items DB exporter generically, land and push it in the constitution submodule per CONST-049, and only then re-cascade to the fleet and bump pointers. Who benefits: every consuming project whose manuals inherit the anchor text, and any agent reading a submodule manual to learn how tracker summaries are produced. Acceptance: canon names the DB exporter in project-neutral wording, the change is pushed to every constitution upstream, all 129 submodule copies are re-cascaded, submodule pointers are bumped, and the cascade verifier plus CM-SUPERSEDED-GENERATOR-NAMING both pass with scope widened to the fleet.

HXC-188 — The project handbook describes a folder layout the project no longer has

Status: Queued **Type:** Task **Severity:** Low **Created-By:** Claude

The main handbook that tells contributors and automated helpers how this project is organised lists several folders at the top level that do not exist. It names four internal areas when only one is present, and refers to a top-level commands folder that is absent entirely. Anyone following it — a new contributor, or an automated helper reading it for orientation — will look for things that are not there, and may conclude the project is broken or that they have checked out the wrong thing. It also undermines trust in the rest of the document, which is otherwise the authoritative description of how to work here. The correction is small and purely descriptive: measure the actual layout and update the section to match, then keep the two in step as the layout changes.

HXC-189 — A dependency is pinned to an old project name that only works because of a redirect

Status: Queued **Type:** Task **Severity:** Low **Created-By:** Claude

One of the external projects this codebase depends on has been renamed. Our configuration still refers to it by its previous name, which currently works because the hosting service silently forwards requests from the old name to the new one. That forwarding is a courtesy, not a guarantee: it disappears if anyone ever creates a new project under the old name, and it is not honoured by every tool that might fetch the dependency. If it stops working, fetching the project fails with an error that points at a name nobody recognises, which is unusually confusing to diagnose. The fix is to record the project's current canonical name so the reference stands on its own rather than depending on a redirect continuing to exist.

HXC-190 — Commands stopped by a timeout are reported as though they were not

Status: Queued **Type:** Bug **Severity:** Low **Created-By:** Claude

When a running command is cut short because it took too long, the result that comes back says it was not a timeout. The check that sets that flag looks at the wrong clock: it inspects the caller's own deadline, which usually has none at all, rather than the timer that actually stopped the command. Investigation showed the obvious alternative would not work either, because the internal signal it would consult can only ever report a plain cancellation and never a timeout, and the timer that does the stopping is a separate mechanism entirely. The effect is that anyone diagnosing a slow command sees a generic failure and no indication that a time limit was the cause, which sends them looking in the wrong place. A sibling code path that runs commands the ordinary way sets the flag correctly by doing it inside the timer's own callback, so a working pattern already exists to copy.

HXC-191 — Cancelling a command cannot actually stop it when the protective sandbox is switched off

Status: Queued **Type:** Bug **Severity:** Low **Created-By:** Claude

When a command is cancelled or times out, the system tries to stop not just that command but anything it started, by signalling the whole group of processes together. That only works if the command was placed into its own group when it began. With the protective sandbox enabled — the normal configuration — it is, and cancellation works. With the sandbox switched off, the grouping step is skipped but the stop attempt still asks the operating system to signal a group that does not exist, which fails silently because the error is discarded. The result is a cancelled command that keeps running. Today this is dormant, because the only configuration that disables the sandbox is used by a single test, so no shipped path reaches it. It is worth fixing because the failure is completely silent and would appear only in an unusual configuration, which is the hardest kind of problem to diagnose later.

HXC-192 — A single over-long line of output permanently stops reading, which can freeze the command producing it

Status: Queued **Type:** Bug **Severity:** Low **Created-By:** Claude

Output from a running command is read line by line, with a limit on how long any single line may be. When a line exceeds that limit the reader stops — not just for that line, but permanently for that stream, because the buffer is fixed at the limit and can never grow past it. Nothing else takes over the reading. The command on the other end keeps writing until the operating system's own small buffer fills, and then it blocks forever waiting for someone to read. So one unusually long line, which programs emitting compact machine-readable output produce quite naturally, can wedge the command that produced it. The fix is to keep draining and discarding the remainder of an over-long line rather than abandoning the stream, so the producing command is never starved of a reader.

HXC-196 — The project handbook names a component version the code no longer uses

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude

The handbook that describes this project's technology stack names a specific version of one of its networking libraries. Both parts of the codebase now use a newer version, because a security fix required moving off the older one. The upgrade was correct and deliberate; it is the handbook that is now wrong. This matters because that section is what a new contributor or an automated helper reads to learn what the project runs on, so it will confidently state a version that has not been used for some time — and because the same document has already been found describing folders that no longer exist, each additional inaccuracy makes the whole document less trustworthy. The fix is to update the recorded version to what is actually in use, and to note that the security upgrade is the reason, so nobody later reverts it believing the handbook was correct.

HXC-197 — A directory tree contains a stale copy of itself nested several levels deep

Status: Queued **Type:** Bug **Severity:** Low **Created-By:** Claude

One test directory contains a duplicate of its own path nested inside itself, so the same ten files appear twice in the project at two different depths. It was created by an earlier renaming exercise that moved a tree while leaving a copy behind at the old location inside the new one. Nothing reads the nested copy, so it causes no failure today, but it

means anyone searching the project finds two results for every one of those files and cannot tell which is real, and any tool that scans for duplicate declarations reports it forever. It also blocks a useful check from being switched on: a guard that detects accidentally-nested duplicates cannot be enabled while this one exists, because it would report a failure on every run. Removing it requires first confirming through the project history that the nested copy is genuinely the leftover and not the original.

HXC-200 — A leak detector recognises only one of the two shapes the leak can take

Status: Queued **Type:** Bug **Severity:** Low **Created-By:** Claude

A detector was written to catch a specific kind of stall where a background worker gets stuck trying to hand off a result nobody is collecting. It identifies that stall by looking for one particular description of what the worker is waiting on. There is a second, equally real form of the same stall — where the worker waits on a choice between several possibilities rather than a single handoff — and the detector does not recognise it. This was demonstrated rather than theorised: a deliberately introduced leak of the second form went completely undetected while the detector reported everything healthy. The shape it does catch is the historical one, so the guard is not useless, but a future change that keeps the newer structure while breaking its escape route would slip past. Widening it to recognise both descriptions is a small change and restores the guard to covering the whole defect rather than half of it.

HXC-204 — Two more endurance tests report failure whenever the machine is busy

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude

Two long-running tests — one exercising heavy simultaneous access to the memory store, one exercising leader election among worker nodes under deliberately induced faults — report failure when the machine is under load, and pass comfortably when run on their own. Measured: during a full test run both failed, while in isolation they passed three times consecutively, one finishing in roughly a fifth of its allowed time and the other in about a twentieth. Their verdicts are therefore reporting how busy the machine was, not whether the product works.

This matters most at exactly the wrong moment: the complete test run performed before a release is by definition the busiest the machine ever gets, so these two will mark a healthy release as broken and train everyone to wave the result through. A third test with the same problem was already corrected by making an inconclusive run report as skipped rather than failed, while keeping its real checks strict; the same treatment fits here. Both should wait for the condition they care about rather than assuming a fixed time is enough.

HXC-206 — A provider stopped and restarted during a health probe can have a stale verdict applied

Status: Queued **Type:** Bug **Severity:** Low **Created-By:** Claude

The newly-fixed health refresh deliberately releases its lock while it contacts each provider over the network, so that a slow or hanging provider cannot block everyone else. It then re-acquires the lock and only applies its verdict if the provider's state has not changed underneath it. There is a narrow remaining window: if a provider is stopped and started again entirely within one probe, the check sees the same state it started with and applies a verdict formed against the previous instance. The result would be a provider briefly reported unhealthy when it is fine, which the next refresh corrects, and the dangerous direction — reporting a stopped provider as healthy — is already prevented. Worth recording that this is cheap to close rather than expensive: a hidden counter incremented on each state change is invisible to the outside world and requires no change to any published structure. An earlier note describing it as a published-interface change was wrong and has been retracted.

HXC-207 — Every model manager shares one settings map by reference, so a future write would corrupt all of them

Status: Queued **Type:** Bug **Severity:** Low **Created-By:** Claude

When a model provider record is created, its environment settings are taken from a shared template by reference rather than copied. Every record in every manager therefore points at the same underlying settings, including across separate manager instances. Nothing writes

to those settings today, and the values handed out to callers are copied, so no corruption occurs at present — this was verified rather than assumed. The reason to record it is that the protection now added to these managers is per-instance: a lock in one manager cannot protect a structure shared with another. So the day someone adds a legitimate, properly-locked write to a provider's settings, it will silently alter every other manager's view as well, and the locking will look correct while failing. Copying the settings when the record is created removes the hazard permanently and costs nothing.

HXC-208 — Two mock services are wired into e2e compose stacks that cannot build them

Status: Queued **Type:** Bug **Created-By:** Claude **Severity:** Low (test-infrastructure only — the Slack and LLM-provider mocks are e2e test doubles, never shipped to users; blocks container-level proof for those two mocks specifically, but the concrete permissive-origin fix that surfaced this gap was still fully provable and shipped at the source/process layer)

Four compose entries across two end-to-end test stacks declare a build context pointing at the Slack mock and the LLM-provider mock, but neither directory contains a Dockerfile. Any attempt to build those stacks therefore fails at the build step rather than at run time, which means the containerised form of these two services has almost certainly never been exercised. This was surfaced while fixing an unrelated permissive-origin defect in the same two services: the fix could be proven at the source and test-process level but not against a running container, precisely because no container of either service can be produced. That gap matters beyond these two mocks, because a fix validated only in-process leaves the deployed-artifact layer unproven, which is exactly the class of silent failure the four-layer verification discipline exists to catch. The work is to add the missing Dockerfiles, or to remove the build entries if these services are genuinely never meant to run containerised, and then to prove the choice by building the stack.

HXC-216 — Thirty-five historical evidence folders hold runs that were never saved to the repository

Status: Queued **Type:** Task **Created-By:** Claude **Severity:** Low (the recurrence-causing gitignore bug is already fixed — commit 7552c7bd; I confirmed only 1 untracked docs/qa/ dir remains today, not 35; remaining scope is a one-time backlog cleanup of already-closed items' historical evidence, with no bearing on any currently open or future closure)

A blanket rule excluding log files from version control was also excluding captured test evidence, because evidence transcripts are written as log files. The rule has now been corrected so this cannot happen again, and the three folders backing recently closed items have been recovered and committed. Thirty-five older folders remain in the same state: the runs exist only on the machine that produced them, so anyone cloning this project sees an evidence folder that appears empty, which is indistinguishable from a test that was never run. Recovering them is not automatic, because two of the files alone account for seventy-eight megabytes and committing those into permanent history is a poor trade that deserves a deliberate decision rather than a reflex. The work is to review the thirty-five, keep what genuinely substantiates a closed item, trim or summarise the two very large ones rather than storing them whole, and record explicitly which ones are being let go and why, so the gap is visible rather than silent.

HXC-219 — Evidence-gate handbook never explained the new citation rule, and its exported copies are stale

Status: Queued **Type:** Task **Severity:** Medium

A recent repair changed how the release gate decides whether a piece of recorded test evidence really belongs to the change it claims to document. Evidence must now name its commit on a labelled line rather than merely happening to contain the right code somewhere in the text. That new requirement was never written down for the people who have to follow it: the guide that authors read before recording evidence says nothing about the new labelled field, so a contributor following the current instructions can produce evidence the gate will

reject without understanding why. Separately, the reference page describing the gate itself has printable and web copies that were generated well before the page was last edited, so all three versions disagree with each other and with the tool they describe. Fixing this means updating the author-facing guide to describe the labelled citation field and regenerating the exported copies so every version matches. The people who benefit are contributors recording evidence and reviewers reading it, who currently have no accurate written description of the rule being enforced. Success means someone can follow the written guide and produce evidence that passes the gate on the first attempt.

HXC-225 — One vendored helper server accounts for two thirds of all our reported vulnerabilities

Status: Queued **Type:** Task **Created-By:** Claude **Severity:** Medium (direct sibling of HXC-171, same shape and same rating: a vendored component of unverified origin/usage inflates the headline vulnerability count — 140 of roughly 208 — burying the few that matter; not Low because an undiscussed component silently shipping or wired in unnoticed is a real provenance/supply-chain gap requiring an explicit operator decision; not High because HXC-166's independent triage already found 0 of 210 total advisories reachable-and-unmitigated in any shipped helix_agent artifact today)

A single bundled helper component inside the agent project is responsible for 140 of the roughly 208 security advisories reported against the whole project — about two thirds of the total — and it entered the codebase without ever being discussed. Nobody has established what it does, whether we wrote it or copied it in, whether anything we ship actually uses it, or whether removing it would cost us anything. Until those questions are answered the headline advisory count is misleading: it reads as though the agent project carries enormous risk, when in reality the great bulk of it sits in one component that may not even be part of what we deliver. The work is to determine the component's origin and purpose, whether it is reachable from anything we build or run, and then to make a decision about it — keep and maintain, upgrade its dependencies, or remove it — and record which was chosen and why. The decision matters more than the individual advisories, because upgrading dependencies inside a component we do not need would be effort spent on nothing.

HXC-224 — Closure-evidence pointers are unchecked free text, so proof of completed work silently becomes unreachable

Status: Queued **Type:** Bug **Severity:** High **Created-By:** Claude
Evidence: docs/qa/hxc217_evidence_path_resolve_20260805T082033Z

When we mark a piece of work finished, we record a pointer to the proof that it really works. Nobody ever checked that those pointers actually lead anywhere. As a result three different kinds of rot accumulated silently and none of them were visible to any test or report. First, some pointers hold a whole paragraph of explanation instead of a location, so there is no way for a machine to follow them. Second, some point at a folder that was never created because the person doing the work saved the proof somewhere else, which means the proof exists but nobody can find it from the record. Third, some are attached to ordinary progress updates rather than to the actual completion, which makes routine notes look like formal proof. The effect is that a completed item can look fully evidence-backed in every summary and dashboard while the evidence is unreachable, and the only way anyone finds out is by manually hunting for it long afterwards. The work here is to make the completion pointer a checked field: it must point at something that genuinely exists, it must be attached only to real completions, and anything that fails those rules must be refused at the moment someone tries to record it rather than discovered months later. This protects everyone who relies on our finished-work reports being true, because a claim of proof that cannot be produced on demand is worse than no claim at all.

Forensic anchor (FACT, HXC-217 audit 2026-08-05)

Of 127 closure records carrying an evidence pointer, **16 (across 14 tickets) did not resolve**. Systematically resolving every one:

- **4 rows** (HXC-107, HXC-108 x2, HXC-112) were `event_type='Updated', reason='operator-blocked'` **progress notes, not closures**. Their real closure events resolve cleanly. These were false positives of the audit's own population query — a guard that refuses a valid state (§11.4.201(1)).
- **4 rows** (HXC-124, HXC-131 x2, HXC-132) were **path drift**: substantive git-tracked evidence exists at `scratch/discovery/fixes/*_evidence.md`; the DB recorded a `docs/qa/...` path that was never populated. An earlier pass concluded these files "never existed"

because it searched the recorded (wrong) full path instead of the basename across full history.
- 7 **rows** (HXC-158, 161, 165, 167, 182, 184, 203) held **narrative in the path field**; every referent proved producible and has been repointed.
- **1 row** (HXC-187) had **no producible artefact at all** — commit references only. Reopened separately under §11.4.34.

Closure criteria

1. `workable-items validate` refuses a closure whose `evidence_path` does not resolve, scoped to genuine closure events so it cannot false-positive on progress notes.
2. A paired §1.1 mutation proves the guard is not a tautology.
3. The population query distinguishes closure events from historical updates. ## HXC-226 — The copy of the agent project we actually pull from reports no vulnerabilities at all

Status: Queued **Type:** Bug **Created-By:** Claude **Severity:** Medium (independently verified via GitHub GraphQL: vasic-digital/HelixAgent=210 open alerts, HelixDevelopment/HelixAgent (our submodule's own configured origin per .gitmodules)=0, identical commits; a monitoring blind spot on the copy we actually consume, not a live production vulnerability, since HXC-166's independent triage found 0 of 210 reachable-and-unmitigated in any shipped artifact today; not Low because a future genuinely dangerous CVE would land completely unnoticed on the consumed mirror until this is fixed, and any current zero-findings claim about this component is unfalsifiable)

The agent project is mirrored to two hosting accounts, and automatic vulnerability alerting was only ever switched on for one of them. Queried today, one mirror reports two hundred and ten open alerts while the other reports none. The one reporting none is the one the main project is configured to pull from, so anyone who checks the security posture of the component we actually consume is shown a clean result while more than two hundred findings sit unseen on the sibling copy. Nothing is wrong with either set of code; the two mirrors hold the same commits. What differs is that only one of them is being watched. This is dangerous precisely because the answer looks reassuring: a report of zero findings is indistinguishable from a report of zero findings because nobody is looking, and the second case is what we have. The work is to switch alerting on for the mirror we pull from, confirm both then report the same figures, and add a periodic check

that compares the two so a future divergence is noticed rather than trusted. Until that is done, any statement about this component's vulnerability status must name which mirror it came from.

HXC-227 — A working access key for an external service is published in a design document

Status: Queued **Type:** Bug **Created-By:** Claude

A design document committed forty-eight days ago contains, in its API-keys section, what appears to be a genuine access key for an external model provider rather than a placeholder. The section is headed as being copied from the machine's own credentials file and described as already configured, and the value carries none of the markers a placeholder would have: it is fifty-one characters long, uses twenty-nine distinct characters, and contains no example or to-be-filled wording. The document is tracked on the main branch and has therefore been published to all four hosting mirrors since the day it was written. Anyone with read access to any mirror, and anyone holding a clone or fork made at any point in those forty-eight days, already has the value. Deleting the line now would remove it from the current revision only; the history retains it, rewriting that history is forbidden, and rewriting would in any case not reach copies already distributed. The only action that actually withdraws the key is revoking it at the provider and issuing a replacement, which requires access to the provider account and cannot be done from within the repository. Until that happens the key must be assumed compromised. Once revoked, the document should be edited to reference the credentials file by name instead of quoting its contents, and a check added that refuses any commit introducing a value of this shape.

HXC-231 — HelixLLM gateway reports itself as running for over half an hour while refusing all connections

Status: Queued **Type:** Bug **Created-By:** Claude

The HelixLLM gateway downloads large AI model files from an external website before it starts accepting network connections. During that download the operating system reports the service as

running and healthy, but every attempt to reach it fails outright. Measured on 2026-08-06 this window lasted thirty-seven minutes from service start until the service actually began listening. Anyone checking the service status during that period is told everything is fine while the service is in fact unusable, and any other component that depends on the gateway will fail for the whole window. The problem is made worse because the download depends on an external site being reachable and fast: on this run one of the two downloads timed out after roughly thirty minutes and was abandoned. If that site were unreachable the gateway might never start listening at all. The fix is to start accepting connections first and download models in the background, and to report an honest not-ready state while the downloads are still in progress.

HXC-232 — Two database tables the software expects are missing, so HelixAgent logs an error every minute forever

Status: Queued **Type:** Bug **Created-By:** Claude

HelixAgent expects two tables to exist in its database, named distributed_locks and agent_instances, but neither of them has ever been created. There is no setup script anywhere in the project that creates them. As a result HelixAgent tries to tidy up expired locks once every minute, the database refuses the request because the table does not exist, and an error is written to the log. Verified on 2026-08-06 by checking all three databases directly: neither table exists in any of them, and the error appeared sixty times in a sixty-minute period plus a further hundred and thirty times during the boot test. The service otherwise runs normally, so nothing visibly breaks, but the constant stream of errors buries genuine problems in the log and means the locking feature these tables were meant to support is silently doing nothing at all. Fixing this needs someone who knows what these tables were intended to contain to write the setup script; the columns must not be guessed.

HXC-234 — HelixAgent’s plug-in tool servers fail to build on startup and never become available

Status: Queued **Type:** Bug **Created-By:** Claude

On startup HelixAgent tries to build and launch a set of optional plug-in tool servers, known as MCP servers, using the container engine. The build fails: two of the packages inside the container fail during their dependency-install and compile step and the whole operation is abandoned. HelixAgent records a warning saying it failed to start some of these servers and then continues running normally, so the platform looks healthy and nothing obviously breaks. The real effect is that every capability those plug-in servers were meant to provide is silently missing, and because the failure is only a warning it is easy for nobody to notice. Observed live on 2026-08-06 during a full platform boot, where the build was attempted twice and failed both times after downloading and unpacking a large container image each time, which also makes every startup slower. The fix requires repairing the dependency installation inside those container build files.

HXC-235 — Search-by-meaning quietly falls back to a method that cannot understand meaning

Status: Queued **Type:** Bug **Created-By:** Claude

The HelixLLM gateway provides a feature that finds documents by meaning rather than by exact wording. On startup it reports that no real language model is configured for this purpose, so it has fallen back to a simple hashing method. The gateway’s own warning states plainly that this fallback does not capture meaning at all and that retrieval quality will be significantly degraded. The important part is that the feature does not switch itself off or return an error: it keeps answering every request, just with results that are effectively arbitrary rather than relevant. Anyone using it would have no way of telling from the responses that it is not working properly. Observed live on 2026-08-06 during a full platform boot. Resolving this means pointing the relevant configuration setting at a genuine embedding provider, and deciding whether returning meaningless results silently is acceptable behaviour or whether the feature should refuse to answer instead.

HXC-236 — Address parsing behaves differently in two parts of the codebase, and will change silently on a version bump

Status: Queued **Type:** Bug **Created-By:** Claude

The standard library routine that splits a web address into a host and a port behaves differently in two parts of this codebase, using the same compiler and the same input. In the main application it rejects an address whose host is an unbracketed IPv6 literal; in the containers component it quietly accepts the same string and guesses the port from the last colon. The cause is that each component declares which language version it targets, and the newer version tightened this routine; the containers component still declares the older one, so it keeps the older, permissive behaviour. Two consequences follow. First, any reasoning about address handling is only valid for the component it was measured in, which has already caused one incorrect conclusion to be recorded during this work. Second, and more seriously, the day someone raises the containers component to the newer language version, addresses it accepts today will start being rejected, and the failure will appear as unreachable services rather than as anything pointing at the version change. The work is to make the difference explicit rather than incidental: decide deliberately which behaviour each component should have, record that decision where a reader will find it, and add a check that fails if the two drift apart again without anyone noticing.

HXC-238 — Captured proof files are trusted as harmless because of their name, not because anything stops them running

Status: Queued **Type:** Bug **Severity:** Low

The safety check that stops half-finished experiments from being committed now makes an exception for captured proof files, which was necessary because those files can never satisfy the old exception rule and so could not be saved at all. The exception is deliberately narrow: the file must sit in the evidence folder, carry one of four harmless file types, be saved as non-runnable, and not begin with the line that marks a file as a program. Together those make the file inert in normal use. What none of them prevent is somebody deliberately

pointing a program interpreter at the file and running its contents anyway, which no file property can stop. The practical risk is low, since it requires a deliberate act rather than an accident, and the alternative was leaving the project unable to save the proof it is required to keep. Recording it so the exception is understood as bounded rather than absolute, and so anyone widening it later can see what it never claimed to cover.

HXC-239 — HelixQA http runner defaults to the wrong login token field, so every authenticated test bank fails to log in

Status: Ready for testing **Type:** Bug **Created-By:** Claude

When HelixQA runs a test bank that needs to sign in, it cannot find the access pass in the sign-in reply, so every test in that bank that needs to be signed in fails. CORRECTED MECHANISM (the original wording of this item was wrong, and the correction matters): the reply carries the real access pass at the top level under the name token, while the separate bookkeeping reference for the session sits NESTED one level down inside a session object. The runner looked for the bookkeeping name at the top level only, found nothing there, and stopped with a plain complaint that the field was missing. It therefore never handed the server a wrong value and was never turned away for using one — it failed before making a single signed-in request. The original text described it as picking the wrong value and being rejected, which is not what happens; the reported counts in that text were nonetheless correct. A SECOND PLACE FAILED SILENTLY: the API validation step had its own copy of the same assumption with no way to override it, and when it could not find the pass it simply carried on without one, so every later request went out unsigned and the resulting rejections were recorded as faults in the service being tested rather than as a failure to sign in — which is why this looked like a broken service. WHY THE OBVIOUS FIX WAS WRONG: the bookkeeping name is the CORRECT name for a different family of about fourteen banks that target a separate catalogue service, whose access pass genuinely does sit at the top level under that name. Simply swapping the two names would have repaired one product by breaking every catalogue bank. The fix keeps the existing name first and adds fallbacks tried only when it is absent or empty, plus support for reaching a nested location, and reports clearly when nothing usable is found. HOW TO REPRODUCE: run a bank requiring sign-in against the platform and

observe the run stop with a missing-field complaint before any signed-in request is made. MEASURED OUTCOME: the bank used for measurement went from six passing and four failing to eight passing and two failing. The two that remain are unrelated to signing in and pre-date this work: one posts an empty body where the service expects content, and the other collides with a leftover record created by the very measurement run that produced this item's original figures, because that bank does not clean up after itself. That leftover is also why the originally reported nine-and-one is no longer reproducible. ACCEPTANCE CRITERIA: banks needing sign-in authenticate against both service families without configuration changes; a failure to find a usable pass is reported as its own explicit finding rather than as service faults; no credential value ever appears in a message or log; and a standing test proves the behaviour, shown real by a paired mutation that makes it fail.

HXC-240 — The generate-e2e test bank can only ever be run once because it registers a fixed user name that then already exists

Status: Queued **Type:** Bug **Created-By:** Claude

One of the HelixCode test banks signs up a new account as part of its checks, but it always uses the very same account name, 'hxc_gen_e2e'. The first time the bank is run the sign-up succeeds and the check passes. Every run after that hits the same account already sitting in the database, the server correctly refuses with 'user already exists', and the check is recorded as a failure forever after. This was observed live within a single session: the first run of the bank passed that step, and a second run minutes later failed it, with the account visible in the database timestamped to the moment of the first run. The rules this project works under require that automated checks can be re-run any number of times and give the same answer, cleaning up after themselves. As written this bank breaks that rule and quietly poisons its own results, so a permanent red mark accumulates that has nothing to do with whether the product works. The fix is for the bank to use a fresh unique name each run, or to remove the account it created when it finishes.

HXC-241 — Test bank reports a fully working AI text-generation feature as broken because it matches wording with the wrong capital letters

Status: Queued **Type:** Bug **Created-By:** Claude

The checks that confirm HelixCode can generate AI text look for an exact run of characters inside the server's reply, and that comparison treats capital and small letters as different. Two checks in the generate-e2e bank ask for wording that the server does produce but with different capitalisation, so they are marked as failures even though the product behaved correctly. The most serious case asks the AI to say hello and then searches the reply for 'hello' in small letters; the AI answered 'Hello' with a capital H, so a genuinely working feature was reported as broken. This was confirmed by hand against the live server: asking it to reply with an invented word returned that exact word back, and asking it what seventeen times three is returned fifty-one, both with believable token counts, which proves real AI generation is running and healthy. A second check searches an error reply for 'authorization' in small letters while the server writes 'Authorization' with a capital A. Expecting exact capitalisation of freely generated AI wording is unreliable by nature. The consequence is false alarms that hide real problems and waste investigation time.

HXC-242 — Screenshot and QA-session features are switched off in the running deployment so five checks cannot be exercised at all

Status: Queued **Type:** Bug **Created-By:** Claude

Five checks covering the built-in screenshot and quality-assurance session features fail against the running HelixCode server, and every one of them fails for the same reason: the server answers 'QA engine is disabled' and reports itself temporarily unavailable. The server is being honest here rather than misbehaving, but the practical effect is that a whole advertised area of the product is switched off in this deployment and therefore cannot be used by anyone or confirmed as working by anybody. The affected areas are listing the available

screenshot engines, listing quality-assurance sessions, asking for the status of a session, starting a new session, and the handling of a request to start a session with nothing to run. Because the feature is off, we have no evidence either way about whether the underlying code behind it functions. Someone needs to decide whether this feature is meant to be available, and if so switch it on and re-run these checks so the behaviour is actually proven; if it is meant to stay off, the checks should say so rather than reporting failures.

HXC-243 — Some of our own test suites cannot fail, so their passes mean nothing

Status: Queued **Type:** Bug **Created-By:** Claude

Several of the automated test collections that check our services declare no expectation about what a correct answer looks like. They send a request, receive whatever comes back, and record a pass regardless of the reply. This was demonstrated rather than argued: one collection was deliberately aimed at the wrong service, one already proven to be returning an error for every request, and it reported both of its checks as passing. A collection that passes against a service known to be broken cannot tell a working feature from a dead one, so its green result carries no information at all. This matters more than an ordinary failing test, because a suite that cannot fail is worse than no suite: it produces confidence where there is none, and it will keep producing it every time it runs. The work is to give every check an explicit expectation — the status it should return, or the content the answer must contain — and then prove each one is capable of failing by pointing it at something known to be wrong and confirming it reports a failure. A separate finding from the same run: four apparent failures in the worker collection were traced to the test harness looking for the wrong field name in the login reply, not to any fault in the product, and that mismatch should be corrected so real failures are not lost among false ones.

HXC-245 — Regenerating the tracker documents from the database silently destroys the blocked-item explanations

Status: Queued **Type:** Bug **Severity:** High

When the workable-items tool regenerates the human-readable tracker documents from the database and then reads them back, the explanations attached to blocked items are lost. Measured on 2026-08-08: the database holds five such explanation records; after one regenerate-and-reread cycle only one survives, so four are destroyed. The cause is that the document generator never writes these explanations into the document in the first place, so when the document is read back there is nothing to restore from. This matters because those explanations are the entire reason a blocked item is actionable — they record what is blocked, what was already tried, and exactly which decision would unblock it. Losing them turns a tracked decision waiting on a person into an item nobody can act on, and it happens silently, with no warning and no error. It also breaks the guarantee that the database and the documents are two faithful views of the same data, which is the foundation the whole tracking system rests on. Anyone maintaining the project benefits: the operator keeps the context needed to make blocking decisions, and future contributors can see why an item stalled. The expected outcome is that a regenerate-and-reread cycle preserves all five explanation records, proven by a test that fails today and passes after the fix.

HXC-246 — The agent runtime’s automated test suite reports 74 failures, and most look like the tests, not the product

Status: Queued **Type:** Bug **Severity:** High

A complete run of the agent runtime’s automated tests finished with 366 groups passing, 12 groups failing, and 74 individual test failures. The failures are being investigated rather than assumed, because the early evidence points at the tests and their environment rather than at the product. Three patterns account for most of them: a large group cannot reach the in-memory cache service because they look for it on the port number it uses INSIDE its container while this machine publishes it on a different one; a second group expects a running service to answer a web request and fails instantly, even though that service is up and answering correctly when asked by hand; a third group measures elapsed time against fixed limits and overshot them by about a quarter, having run while four other jobs and a service rebuild were competing for the same processor. None of that proves the product is healthy, and none of it proves the product is broken - that is exactly what four parallel investigations are now establishing, each

required to produce captured evidence rather than a plausible story. This matters because a test suite that fails for environmental reasons is as damaging as one that passes while the product is broken: both teach everyone to stop trusting the result, and both hide the real defects in the noise. The operator benefits from a suite whose red means something, and future contributors get a signal they can act on. The expected outcome is every one of the 74 failures classified with evidence as a genuine product defect, a test defect, an environment coupling, or a contention artifact, each real defect fixed with a test that reproduces it first, and the suite returning to a trustworthy state.

HXC-247 — Two services both claim network port 8100, so 82 test files test the wrong program

Status: Queued **Type:** Bug **Severity:** High

Two separate parts of the system have both been told to use the same network address, and neither knows about the other. The agent runtimes own address book - the file that decides which service gets which port number - assigns port 8100 to the agent runtime itself. But a different component, the model verifier, has its own configuration that also names port 8100, and because the verifier starts first it takes the address. Meanwhile the agent runtime is launched from a settings file that still names its old address, 7061, so it never even tries to claim the one it was allocated. The visible result is that 82 test files ask port 8100 for the agent runtime and are answered by the verifier, which does not recognise the requests and rejects them with a not-found - so those tests report the agent runtime as broken when it is running perfectly well a few numbers away. This matters because it makes a large part of the test suite structurally incapable of testing the thing it names, and because the same collision would mislead anyone deploying the system for real. Whoever operates or deploys this benefits directly: today a healthy service reads as failing, and a genuinely failing one would look exactly the same. The expected outcome is a single place that decides port allocation for every component, no two components claiming the same number, the agent runtimes own settings agreeing with that allocation, and the tests then passing or failing for reasons that are genuinely about the product. Later evidence settles which of the two sides is the mistaken one, and it turns out the deployment is not misconfigured at all. The model verifier’s own service definition - the file that starts it on this machine - documents in its own header that

this address is the agreed meeting point between the verifier and the gateway that consumes it, that the gateway looks there by default, and, in as many words, that the agent runtime actually runs on the other address here. The choice was therefore made deliberately and written down at deployment time, in the verifier's favour. That makes the defect narrower and clearer than first described: the agent runtime's internal address book is simply out of date, still claiming an address that a different, documented owner holds, and annotating it with a note that the old address was abandoned when in fact the old address is the one still in use. A contributing cause is that the verifier has no entry at all in that address book, so the one mechanism meant to prevent two components claiming the same number could not arbitrate a component it had never been told about. The practical consequence is unchanged - the tests inherit the stale claim and are answered by the wrong service - but the repair direction is now settled rather than open: correct the agent runtime's entry to the address it genuinely serves, give the verifier an entry so the collision cannot silently recur, and point the test defaults at the real address. The honest limit is that this establishes which side is stale and why; it does not by itself establish that renumbering is safe to execute, because that entry is consumed across the whole component and the change still needs its impact review before it lands.

HXC-248 — Test cleanup can shut down the live platform; only a coincidence prevents it today

Status: Queued **Type:** Bug **Severity:** High

A cleanup step in the automated test suite can shut down the running platform, and today the only thing preventing that is a coincidence. When the integration tests finish they try to stop any containers they started. To avoid stopping containers they did not start, they first check whether the agent service is still running - but the check is only whether *something* answers on network port 8100, with no verification of what that something is. On this machine port 8100 is held by a different component entirely, the model verifier, so the check says yes, and the shutdown is skipped. The agent service itself is running on a different port and is never actually consulted. If the verifier were ever stopped, restarted late, or moved, that same check would say no and the test cleanup would shut down the live platform out from under whoever was using it. The log line it prints while doing this is also

untrue: it reports that the agent is still running on 8100 when the agent is not there at all. This matters to anyone running the test suite on a machine that also hosts a live deployment, which is the normal case here. The expected outcome is a check that confirms the identity of what is answering - not merely that the port is occupied - so the cleanup makes its decision on the real condition rather than a coincidence, and so the message it logs is true. ## HXC-250 — Generated assistant configs send users to the wrong service, so 46 of 48 assistants get a not-found

Status: Queued **Type:** Bug **Severity:** High **Created-By:** AI **Assigned-To:** Claude

When a user asks the system to write out a ready-made configuration file for their command-line coding assistant, the file it produces points that assistant at the wrong service. The generator fills in a network address that belongs to the model verifier rather than to the agent runtime the assistant is supposed to talk to, so the assistant sends its requests to a component that does not understand them and receives a flat “not found” in reply. From the user’s point of view the assistant simply does not work, with nothing in the error explaining that the address in their brand-new config was wrong from the moment it was written. Of the forty-eight assistants the project documents support for, forty-six are affected; a fix landed earlier covers only two of them, so the shared generator that serves all the rest still writes the wrong address today. This is the most directly user-facing defect in the current set, because it is not an internal test that misleads a developer - it is a file handed to a user with an instruction to use it. The people harmed are exactly the newcomers the feature exists to help: someone setting up their assistant for the first time, following the documented path, and getting a broken result they have no way to diagnose. The expected outcome is that every generated configuration names the address the agent runtime genuinely serves, that all forty-eight documented assistants are covered rather than two, and that a generated config works when the user follows the documented setup without editing anything by hand.

HXC-251 — Our own gRPC service could not start while our own platform was running, and its log still names the old address

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** AI **Assigned-To:** Claude

One of our own services could not start while our own platform was running, because both had been told to use the same network address. The address in question is the conventional default that the wider industry uses for this kind of service, which is precisely what makes it the number most likely to be already taken - and on this project's own machine it was taken, by a supporting database container the platform itself starts. The result was that the service refused to start whenever the platform was up, and any client dialling that same well-known address reached the database container instead, which completed a perfectly healthy-looking connection and then rejected every request. Because the wrong service was alive rather than absent, the checks that skip tests when something is unreachable did not skip: they ran, failed, and reported the failures against our service, which had never been running at all. This was also the underlying cause of twenty test failures that had been attributed to the wrong component. The fix registers the service in the project's central address book so its address is allocated deliberately and can never silently collide again, and it now fails immediately with an actionable message rather than starting somewhere unexpected. What remains open is smaller but still misleading: the startup message printed for the operator still announces the old address, so anyone reading the log is told the wrong place to connect. Whoever runs or debugs the platform benefits, and the expected outcome is that the service starts reliably alongside the rest of the stack and that every message it prints names the address it genuinely bound.

HXC-254 — Connector tests share one network handler, so shutting down one test server breaks other running tests

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** AI **Assigned-To:** Claude

Tests in one area of the codebase interfere with each other because they unintentionally share a single piece of network machinery, so a test can fail for reasons that have nothing to do with the code it is testing. When each connector to an external service is created it is given its own timeout setting but no connection handler of its own, and the underlying library quietly falls back to one shared handler belonging to the whole process. The tests in this area start roughly a hundred and forty-five miniature pretend servers and mark around five hundred cases to run simultaneously; each pretend server, when it shuts down at the end of its own test, politely tells that shared handler to close idle connections - and because the handler is shared, it also severs connections other tests are in the middle of using. The result is a failure that appears in a different connector on almost every run and vanishes when the tests are run one at a time, which is the signature of a problem in the test setup rather than in the product. It reproduces reliably enough to matter: roughly one run in ten fails this way. Developers are the people harmed, because a suite that fails unpredictably teaches everyone to re-run it rather than read it, and real defects hide comfortably inside that noise. The expected outcome is that each connector is given its own connection handler so shutting one pretend server down cannot disturb any other test, and the suite becomes trustworthy enough that a red result means something is genuinely wrong.

HXC-255 — Validation runners show green on file existence and never fail the run, which needs an operator decision

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** AI **Assigned-To:** Claude

Two things are left over in the scripts that run the project’s full validation, and one of them needs a decision from the operator rather than a quiet fix from us. The first is a display problem: the end-of-run summary prints “Integration Tests: Completed” in green whenever the log file merely exists, regardless of what the log actually says, so a run in which tests genuinely failed still shows a reassuring green line in the summary a person is most likely to read. The second is deliberate and more consequential: both runners are written to note test failures as warnings and carry on, finishing with an overall success status even when real tests have failed, which means neither script can serve as a gate that blocks a release. That behaviour was chosen on purpose so

that a single failure does not abort a long run and lose the rest of the information, and there is a genuine trade-off here between gathering everything in one pass and refusing to proceed when something is broken. Because it is a deliberate posture rather than an oversight, changing it is an operator decision about how the project wants its release gate to behave, and it should not be altered silently by whoever happens to touch the file next. Whoever depends on these runs to judge release readiness benefits from settling it, since today a green finish means only that the script reached the end. The expected outcome is that the summary line reflects the real result rather than the presence of a file, and that the operator makes an explicit, recorded choice about whether these runners should fail when tests fail.

HXC-256 — Regenerating the tracker documents without naming a destination writes an untracked copy and leaves the real ones stale

Status: Queued **Type:** Bug **Severity:** Low **Created-By:** AI **Assigned-To:** Claude

The tool that regenerates the project's tracker documents writes them next to wherever it happens to be run from unless it is explicitly told otherwise, so an operator who omits that one optional setting quietly produces a second, untracked copy of every tracker document while the real ones stay out of date. The tool's own usage line presents the destination as optional, which is what invites the omission, and its built-in default is simply the current folder rather than the folder where this project actually keeps those documents. This was found the hard way during routine work: running the documented command without that setting created sixteen stray copies at the top of the project while the genuine documents, one directory down, kept their old contents. The consequence is a split-brain state where someone can look at freshly written files, see today's content, and reasonably conclude the records are current when the versions under version control are not. Anyone regenerating the tracker documents by hand is exposed, which in practice means maintainers and automated helpers working outside the standard scripts. The exposure is genuinely bounded and that bounds the severity: every wrapper script the project ships passes the destination explicitly, the tool prints the full path of everything it writes so the mistake is visible to a careful reader, the stray copies show up as new untracked files, and the separate

consistency check would eventually notice the real documents had gone stale. What is missing is the tool itself objecting. The expected outcome is that regenerating the documents without naming a destination either writes them where the project keeps them or refuses and says so, instead of succeeding somewhere else. The check that would have caught this does not exist today: the existing guard for a closely related problem always supplies the destination explicitly, so it can never exercise the case where the setting is left out - adding that second case is the specified follow-up.

HXC-257 — The tracker consistency check calls its input the committed database but never reads version control

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** AI **Assigned-To:** Claude

The automated check that guards our workable-items records promises more than it actually measures, so a reader who trusts its wording is given a guarantee nobody ever computed. Internally it names the thing it inspects “the committed database” and its written description states that it proves the committed database is valid and not stale, but it never consults version control at all - it simply copies the file sitting in the working folder, which may contain changes that have not been committed to anything. What it genuinely proves is narrower and still useful: that the database and the documents on disk right now agree with each other. The gap was demonstrated live during this session, when the check reported success while the database held seven newly filed records that had not yet been committed - exactly the situation its own wording claims to rule out. Worth stating plainly is that the underlying behaviour is probably correct: the check runs as part of the pre-release sweep, where the point is to confirm the material you are about to commit is self-consistent, so inspecting the working folder is the sensible thing to do, and there is a documented reason for the copy - opening the database directly would modify it as a side effect. On that reading the repair is the naming and the wording, not the logic. Everyone who reads that check’s verdict before approving a release is affected, because the difference between “what is on your disk is consistent” and “what is committed is correct” is exactly the difference that matters when the two have drifted. The misleading name has already spread: the description of the paired test that exercises this check repeats the same

claim. The expected outcome is that the check's name, its description, and the sentence it prints all describe only what it measures, and that if the stronger guarantee is wanted it is added deliberately as a separate comparison against version control. The test that would have caught this does not exist today: the paired test plants a disagreement between the documents and the database, but nothing plants a difference between the committed copy and the working copy and requires the check to object - adding that case is the specified follow-up.

HXC-258 — The tracker database records only one sync direction, so it always reports the wrong one after documents are regenerated

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** AI **Assigned-To:** Claude

Our workable-items database keeps a small bookkeeping note saying which way the last synchronisation ran between the database and the human-readable tracker documents. That note is only ever written by one of the two synchronisation paths - the one that copies documents into the database - and the value it writes is fixed text rather than the direction that actually ran. The path that goes the other way, regenerating the documents from the database, never updates the note at all. The practical effect is that after the documents are regenerated the note still describes an older run in the opposite direction, so anyone reading it to find out what happened most recently is told the reverse of the truth. This matters because deciding the direction of a synchronisation is exactly the judgement that can destroy records if it is made backwards, and this field looks like evidence for that judgement while being incapable of supporting it. The note currently reads as a document-into-database run dated the eleventh of July while the most recent actual run was a database-into-documents regeneration on the ninth of August. The correct repair is in the shared tooling that performs the synchronisation, which must record the direction that genuinely ran on both paths rather than assuming one of them; editing the stored value by hand would be undone by the next run and is therefore not a fix.

HXC-259 — Most closed tracker records carry no history of who closed them or when, and the gap is far wider than two items

Status: Queued **Type:** Task **Severity:** Low **Created-By:** AI **Assigned-To:** Claude

Every workable item is supposed to accumulate a short audit trail recording the moments its status changed - opened, reopened, closed - together with who made the change and what evidence justified it. A recent repair noted in passing that two newly filed items had no such trail. Measurement across the whole database shows the situation is not confined to those two: eighty-one distinct records covering one hundred and forty-five rows have no history entries at all, and all but one of them are items that have already been closed and filed away. In other words the majority of our closed record has no machine-readable account of how it reached that state, which weakens any later attempt to audit a closure, to understand why something was reopened, or to reconstruct a timeline for a review. The cause is that items created by direct database writes rather than through the sanctioned creation command skip the step that opens the audit trail, and historical records imported before the trail existed never had one. This is deliberately not being repaired by back-filling: inventing dates and authors for events nobody witnessed would produce an audit trail that is worse than an absent one because it would look authoritative. What is needed instead is a decision on how to mark these records as genuinely unaudited, and a check that prevents any newly created item from joining them.

HXC-260 — HXC-260: HelixAgent k8s manifests published a dead GRPC_PORT env var and port 9090, so the declared gRPC endpoint could never be reachable

Status: Queued **Type:** Bug **Severity:** High **Created-By:** Claude

The Kubernetes manifests for HelixAgent told everyone that the gRPC service lives on port 9090, and they set an environment variable called GRPC_PORT to make it so. Neither half was true. No Go source in the

repository reads GRPC_PORT, so setting it could never move the port the server listens on; this was proven at runtime, where GRPC_PORT=36999 still bound :8112 while the real variable HELIXAGENT_PORT_GRPC=36999 bound :36999. And 9090 is a port no HelixAgent process ever binds: the gRPC server resolves its port from the internal/ports registry (HelixAgentGRPC, core band offset 112), which is 8112 — confirmed by running the built cmd/grpc-server binary and observing its LISTEN socket on *:8112. The practical effect is that anyone deploying HelixAgent to Kubernetes from these manifests would get a Deployment exposing containerPort 9090, a Service forwarding gRPC traffic to targetPort 9090, and a NetworkPolicy admitting traffic on 9090 — three artifacts agreeing with each other and all pointing at a port nothing listens on, while the port the server would actually bind was blocked. The damage is worse than a wrong number because the manifest LOOKED configurable: an operator who noticed the problem and edited GRPC_PORT would see no change whatsoever, defeating their own debugging. This item covers correcting all four manifests that name the port (deployment.yaml, service.yaml, networkpolicy.yaml, configmap.yaml) onto the registry port and replacing the dead variable with the one the server honours. It also records the honest remaining limitation: the image this Deployment runs is built from cmd/helixagent, which contains no gRPC server at all (that is a separate binary, cmd/grpc-server), so the endpoint is now correct but INERT until a grpc-server container is added to the pod — a follow-up that the corrected manifests make a one-line change instead of a re-derivation. Reproduce by checking out the pre-fix tree and running the polarity test in internal/ports with RED_MODE=1, which asserts the dead variable and containerPort 9090 are present. Acceptance criteria: internal/ports/published_grpc_port_test.go passes with RED_MODE unset (asserting every manifest names the registry port and none sets the dead variable outside a comment) and fails on the pre-fix tree, citing deployment.yaml line 32.

HXC-261 — HXC-261: the gRPC protocol challenge recorded success on every branch and probed a port nobody binds, certifying gRPC healthy without ever contacting a gRPC server

Status: Queued **Type:** Bug **Severity:** Critical **Created-By:** Claude

The challenge script that is supposed to prove HelixAgent's gRPC protocol works was structurally incapable of failing, and it has been reporting green for the entire time it has existed. Two independent faults compounded. First, it resolved its port from an environment variable named HELIXAGENT_GRPC_PORT defaulting to 7062 — a third spelling that no Go source reads (the server honours HELIXAGENT_PORT_GRPC) and a number belonging to no service in the project: not the old hardcoded 50051, not the registry's 8112, not the HTTP port 7061. Second, and far worse, every one of its four probes recorded a successful assertion whether the port answered or not: the port-availability check recorded true when the port was listening AND recorded true, labelled 'optional', when it was not; the unary-call, streaming-call and error-handling checks did the same. The two faults together mean the challenge dialled an address no HelixAgent process has ever bound, found nothing there, and then certified the gRPC protocol healthy. This is the most damaging class of defect in the codebase because it is not merely an untested feature — it is an automated test actively producing false-green results, which is exactly what the anti-bluff covenant exists to prevent, and it masks any real gRPC breakage from everyone reading the suite. It also violated the challenge framework's own documented contract: the framework already ships a record_skip helper whose docstring says never to use record_assertion with status true to mask a missing feature, and the primitive was simply not used. This item covers resolving the port through the registry's own variable name and default, consolidating the four duplicated probe sites into one helper so absence is interpreted in exactly one place, making absence report as an honest SKIP carrying a SKIP-OK ticket rather than success, making a real failed call report as FAILED, and replacing the two-state final verdict (which reported PASSED whenever nothing had failed, including when nothing had been verified) with a three-state verdict that reports SKIPPED when no assertion could be verified. Reproduce with the guard at challenges/scripts/tests/protocol_grpc_fail_open_guard.sh run with RED_MODE=1 against the pre-fix script: it binds and releases a

port to prove it is free, drives the challenge's own probe functions against it, and confirms assertions are recorded PASSED anyway. Acceptance criteria: that same guard with RED_MODE=0 reports zero PASSED assertions against a proven-free port with every probe SKIPPED and SKIP-OK tagged, and the challenge still records a genuine PASSED assertion when a real gRPC server is listening on the registry port.

HXC-262 — A safety check in the live-service test battery occasionally reports a failure that is not real

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude

The project keeps a battery of tests whose whole job is to prove its safety checks actually catch problems, by deliberately breaking things and confirming each check notices. One assertion in that battery — the one covering a service stuck in a restart loop — occasionally reports a problem when nothing is wrong. It was seen once across thirteen consecutive runs, and ran correctly six times out of six when run on its own, so it is intermittent rather than consistently broken. The cause is a timing flaw inside the safety check itself: it asks the system two separate questions, whether the service is running and which process is running it, and between those two questions a fast-restarting service can move on, so the two answers describe different moments and disagree. When that happens the check cannot find a process to examine and reports that it could not verify anything. The practical harm is not a missed defect but a false alarm: work gets blocked on a condition that is not present, and worse, people who see a check cry wolf learn to ignore it, which quietly erodes the value of the one battery specifically built to be trustworthy. The fix is to read both pieces of information in a single query so they always describe the same instant. This was deliberately left unfixed by the reviewer who found it because it changes the sampling behaviour of a check that was itself under review at the time, and such a change deserves its own review rather than being folded into someone else's batch.

HXC-263 — Twelve of the fourteen tests that verify our safety checks are never actually run by anything

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude

The project keeps a folder of tests whose purpose is to verify that its safety checks genuinely work, by deliberately introducing a fault and confirming the corresponding check reports a failure. A test like that only has value if something actually runs it. Counting the invocations at the current revision, only two of the fourteen files in that folder are ever executed by anything; the remaining twelve are written, maintained, and never run. This is the same problem the project already found and fixed once before, when three safety checks turned out to exist but be executed by nothing, and it matters for the same reason: a check nobody runs cannot fail, so it certifies nothing while appearing to provide coverage. The presence of these files is actively misleading, because anyone counting test files sees protection that is not being exercised. A careless count makes it look better than it is — five of the files are mentioned somewhere, but three of those mentions are a comment, an entry in a skip list, and a line of prose in a status document, none of which run anything. Resolving this means deciding for each of the twelve whether to wire it into the release sweep so it runs, or to retire it honestly, and then making the choice visible rather than leaving the file sitting there implying coverage it does not deliver.

HXC-264 — One tracker entry describes three different problems and recommends a fix that was never used

Status: Queued **Type:** Bug **Severity:** Low **Created-By:** Claude

A single entry in the project's work tracker manages to describe the same problem three different ways in three different fields, and the three descriptions do not agree. One field says a required program is not installed on the machine at all, another says the program exists but cannot be found because of where the service looks for it, and a third says no such file exists anywhere on the system. Those are materially different problems with different fixes, so a reader cannot tell from the entry what was actually wrong. The entry also proposes a remedy, giving the service a direct path to the missing program, which is not

what was eventually done: the capability was restored by routing requests to a different service that was already running, and the originally-named program is still absent from the machine today. This matters because the tracker is the permanent record of what was reported and why, and an entry that contradicts itself cannot serve that purpose for anyone reviewing the history later. It is not a case of the documents drifting out of step with the database, which is a problem the project already guards against; the rendered documents faithfully reproduce what the database holds, and the inconsistency is inside the database entry itself. Resolving it means reconciling the three fields into one accurate account that records both what was originally observed and how it was actually resolved, without erasing the original report.

HXC-265 — Twenty-five nested Go modules in helix_agent have no compile path and two more build only in un-gated container images, so breakage inside them is invisible

Status: Queued **Type:** Task **Severity:** Medium **Created-By:** Claude **Assigned-To:** Claude

HelixAgent keeps 36 Go modules nested inside its repository (37 counting the root module). Most of them are never compiled by anything the test suite or any gate runs, which means a developer can rename a package, change a shared type, or break an import inside one and every build and test still reports green — the breakage surfaces much later, or never. That silent-failure surface is what this item tracks, and it matters most for the documentation samples, because the project anti-bluff checklist promises that a documentation example works when copy-pasted and an example nothing ever compiles cannot honour that promise. WHO IS AFFECTED: developers who trust a green build, and readers who copy a documentation example expecting it to compile. THE COUNTS, each verifiable by one command (the quotes are load-bearing — unquoted, the shell expands the pattern before git sees it and reports 2 instead of 36): inside submodules/helix_agent, `git ls-files '*/go.mod' | wc -l` reports 36 nested modules, and `git ls-files 'challenges/codebase/go_files/*/go.mod' | wc -l` reports the 6 that the Makefile build-challenge-modules recipe walks. By directory the 36 are 23 under docs/, 6 under challenges/, 2 under plugins/, 2 under docker/, and one each under Toolkit/, pkg/, and cli_agents/. THE CLASSIFICATION IS PER-MODULE,

NOT AN AGGREGATE — two earlier revisions of this item stated a single wrong total because “is it compiled” cannot be answered by any one command, so each class below carries its own evidence. (1) NO COMPILE PATH AT ALL — 25 modules: the 23 under docs/ and the 2 under plugins/; nothing anywhere compiles them, the three challenge scripts touching the plugins pair only run presence checks (test -f, grep -q), and root-level go build cannot reach any of them because Go does not cross nested-module boundaries. (2) COMPILED ONLY INSIDE UN-GATED CONTAINER IMAGES — 2 modules: docker/acp and docker/protocol-discovery, declared as build contexts in docker-compose.protocols.yml and running a real go build in their Dockerfiles, but no test suite or gate exercises those image builds and compose rebuilds only when the image is absent, so breakage surfaces late and non-deterministically rather than never. (3) FULLY INSIDE THE GATED BUILD — 1 module, pkg/api: it is required and replaced in the root go.mod, imported by cmd/grpc-server/main.go with no build tags, and built for four platforms by ci/scripts/ci-go.sh whose failures increment PHASE_FAILURES and set the exit code. It is NOT part of the problem set and needs no work. Note for future readers: `go list ./pkg/api` prints “main module does not contain package”, which is a true output but answers package-pattern membership rather than whether the module is ever compiled — an earlier revision of this item mistook the one for the other. (4) PARTIALLY INSIDE THE GATED BUILD — 1 module, Toolkit: internal/verifier/startup.go imports Toolkit/Providers/Chutes, which transitively pulls in three more Toolkit packages, so at least 4 of its 21 package directories compile on every root build while roughly 17 remain outside. (5) DELIBERATELY EXCLUDED WITH A RECORDED REASON AND A GUARD — 1 module, cli_agents: its go.mod is the HXC-139 isolation marker whose header documents why the vendored tree must not be compiled, paired with the regression guard at tests/regression/cli_agents_isolation_test.go. It already satisfies this item’s second acceptance arm and is terminal. ACCEPTANCE CRITERIA: every module in classes (1), (2) and the un-compiled remainder of (4) is either wired into a build or compile-check target the test suite actually runs, or deliberately excluded with a recorded reason in the manner of class (5); and a gate fails when a new nested module appears that is neither built nor explicitly excluded.

HXC-266 — Agent client URL builder emits a malformed address for IPv6 hosts because it never brackets the literal

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude **Assigned-To:** Claude

The function that builds the URL clients use to reach HelixAgent formats the address as “http://%s:%d” without ever bracketing the host, so any IPv6 host literal produces a malformed URL. WHY THIS IS REACHABLE RATHER THAN THEORETICAL: the resolver layer deliberately treats the IPv6 loopback ::1 as a valid, dialable host and blesses it, so a configuration that sets the host to ::1 passes every validation and then reaches this formatter. HOW IT MANIFESTS: the emitted address becomes http://::1:7061, which is not a parseable URL — the colons of the address and the colon of the port are indistinguishable to any URL parser, so the client fails to connect with a confusing parse error rather than a clear configuration error. WHO IS AFFECTED: any operator running the platform on an IPv6-only or IPv6-preferred host, and any deployment that resolves the agent host to an IPv6 literal instead of a name. HOW TO REPRODUCE: set the agent client host to ::1, request the client base URL, and observe the returned string is http://::1:7061 rather than the correct bracketed form http://[::1]:7061. ACCEPTANCE CRITERIA: the builder brackets IPv6 literals per RFC 3986 so ::1 yields http://[::1]:7061, IPv4 and hostnames are unchanged, and a regression test covers IPv6 literal, bracketed input that must not be double-bracketed, IPv4, and hostname cases.

HXC-267 — HelixQA stays silent when a sign-in reply cannot be parsed, so unauthenticated runs are still misreported as API faults

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude **Assigned-To:** Claude

When HelixQA signs in to run an API test bank and the server answers with a success code but a reply the runner cannot read at all, the runner goes quiet: it records nothing about the sign-in, carries on making requests with no credentials, and files the resulting rejections as faults in the API being tested rather than as its own failure to log in.

A sibling fix (HXC-239) closed this for the case where the reply is readable data with the credential under an unexpected name — that path now raises a clear finding naming what it looked for. This item covers the door that fix left open. HOW IT MANIFESTS: the run looks like the tested service is broken, when in fact the runner never authenticated; whoever reads the report is sent to investigate the wrong system, which is the same misdiagnosis the sibling fix existed to prevent. WHY IT HAPPENS: the code that raises the explanatory finding is nested inside the branch taken only when the reply parses as structured data, and that branch has no counterpart for when parsing fails, so an unparseable reply skips the reporting entirely. MEASURED BEHAVIOUR (each case run directly against the code): a reply of empty text fails to parse and is silent; a reply of HTML such as a 503 error page fails to parse and is silent; a reply of the literal null parses and correctly raises the finding; a reply of an empty data object parses and correctly raises the finding. So the split is precisely between replies that parse and replies that do not, not between replies that carry a credential and replies that do not. HOW TO REPRODUCE: point the pipeline at a test server whose sign-in route returns status 200 with a body of empty text or HTML, run the API validation step, and observe that no sign-in finding is recorded while the later unauthenticated requests are recorded as API faults. WHO IS AFFECTED: anyone reading a HelixQA report for a service whose sign-in route can return a non-data body, including error pages produced by a proxy or gateway in front of the real service. ACCEPTANCE CRITERIA: an unparseable sign-in reply raises the same class of explanatory finding as an unusable one, naming what was attempted and the fact the body could not be read; the message never contains credential values; and a standing test covers the empty-body and non-data-body cases, proven real by a paired mutation that removes the new reporting and makes that test fail.

HXC-268 — Model verifier emits unusable addresses for IPv6 hosts at fifteen capability-config sites that never bracket the literal

Status: Queued **Type:** Bug **Severity:** Medium **Created-By:** Claude **Assigned-To:** Claude

The model verifier builds web addresses by pasting a host and a port together with a colon, without the square brackets that IPv6 addresses require, so any deployment whose host is an IPv6 address produces addresses no client can read. WHO IS AFFECTED: operators on IPv6-first hosts, and any deployment where the configured host is an address rather than a name. WHY IT IS REACHABLE: the generator stores whatever host it is given with no validation, so an IPv6 value passes straight through to the formatting step. HOW IT MANIFESTS: the emitted address looks like http://::1:8100, which no address parser can read because the colons of the address are indistinguishable from the colon introducing the port; the client then fails with a confusing parsing complaint instead of a clear configuration message, sending whoever debugs it toward the wrong system. HOW MANY PLACES, AND A CORRECTION WORTH RECORDING: the capability-config generator contains FIFTEEN such build sites, one in each of fifteen per-agent generator functions, every one reachable from the single dispatch that selects an agent by name. An earlier revision of this item said three. That was wrong in a specific and instructive way: the fifteen sites use only three distinct format SHAPES (nine ending in a version segment, five plain, one ending in a messages segment), and three example line numbers offered by an upstream report were recorded as if they were the complete set. Shapes were counted as places. Had the earlier figure stood, the acceptance criteria would have certified this closed with twelve sites still broken. HOW TO REPRODUCE THE COUNT: inside the verifier submodule, count occurrences of the formatting pattern in the capability-config generator file; it reports fifteen, while listing the distinct pattern texts reports three. RECOMMENDED FIX DIRECTION: do not hand-bracket fifteen places. Concurrent in-flight work on the submodule is introducing a shared endpoint helper whose address builder brackets IPv6 correctly using the standard library join, including zone handling — but that helper is NOT YET part of any committed state of the submodule, so a fresh clone does not have it and nobody should expect to find it there today. Once it lands, route these fifteen sites through it, which fixes them uniformly and prevents a sixteenth from being written wrong. Coordinate with that work before starting. SCOPE NOTE SO A FIXER IS NOT MISLED: the capability-config generator is not the whole surface. The same unbracketed pattern appears in at least nine further files elsewhere in the submodule, including several per-agent config writers and two unrelated subsystems. Those are outside this item deliberately, but a fixer should know the capability generator is one region rather than the entire problem. ACCEPTANCE CRITERIA: all fifteen sites in the capability-config generator produce properly bracketed addresses for IPv6 literals; plain names and IPv4 addresses are unchanged; an

already-bracketed value is not bracketed twice; a standing test covers the IPv6 literal, already-bracketed, IPv4 and hostname cases and is proven real by a paired mutation that removes the bracketing and makes the test fail; and the count of remaining unbracketed sites in that file is zero, verified by the same command that produced the fifteen.